Experiment 12

Multithreading

code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5
pthread_mutex_t chop[N], room;
void *philosopher(void *arg) {
int id = *(int *)arg, left = id, right = (id + 1) % N, r;
for (r = 0; r < 2; r++) {
printf("Philosopher %d is thinking\\n", id + 1);
pthread_mutex_lock(&room);
pthread_mutex_lock(&chop[left]);
pthread_mutex_lock(&chop[right]);
printf("Philosopher %d is eating\\n", id + 1);
usleep(100000);
pthread_mutex_unlock(&chop[right]);
pthread_mutex_unlock(&chop[left]);
pthread_mutex_unlock(&room);
}
return NULL;
}
int main(void) {
pthread_t t[N]; int id[N], i;
pthread_mutex_init(&room, NULL);
for (i = 0; i < N; i++) pthread_mutex_init(&chop[i], NULL);
for (i = 0; i < N; i++) { id[i] = i; pthread_create(&t[i], NULL, philosopher, &id[i]); }
for (i = 0; i < N; i++) pthread_join(t[i], NULL);
for (i = 0; i < N; i++) pthread_mutex_destroy(&chop[i]);
pthread_mutex_destroy(&room);
```

return 0;

}

Output:

